HDOC Day-46 Assignment

Question

Write a C program that reads the size and elements of a variable-length integer array, nums[]. The array may contain duplicate elements. Pass the array to a function named Bubble_sort() using its address and sort the elements in ascending order using the Bubble Sort technique. After sorting, pass the sorted array to another function named occurrence(). Use pointers to pass the array between functions. The function occurrence() must:

Read an integer target.

Find the first and last occurrences of target in the sorted array.

Display the values and indices of the first and last occurrences.

Display -1, -1 if the target is not present.

1. Problem Understanding

The program accepts the size and elements of a variable-length integer array. The array is passed to Bubble_sort() using its address and sorted in ascending order using Bubble Sort. The sorted array is then passed to occurrence() using pointers. The occurrence() function reads a target value, searches for its first and last positions, and displays the corresponding values and indices. If the target is absent, the program displays -1, -1.

2. Standard Test Cases

S.No.	All Inputs	Expected Output	Status
1	Size = 6 Array: 5 7 7 8 8 10 Target = 8	Sorted: 5 7 7 8 8 10 First occurrence: 8 at index 3 Last occurrence: 8 at index 4	Pass
2	Size = 5 Array: 5 7 7 8 10 Target = 8	Sorted: 5 7 7 8 10 First occurrence: 8 at index 3 Last occurrence: 8 at index 3	Pass
3	Size = 6 Array: 5 7 7 8 8 10 Target = 6	Sorted: 5 7 7 8 8 10 -1, -1	Pass
4	Size = 5 Array: 9 2 9 1 9 Target = 9	Sorted: 1 2 9 9 9 First occurrence: 9 at index 2 Last occurrence: 9 at index 4	Pass

3. Algorithm

1. Start.

2. Read the size of the array.

3. Declare a variable-length integer array nums[size] and read its elements.

4. Pass nums and size to Bubble_sort() using the array address.

5. In Bubble_sort(), compare adjacent elements and swap them whenever the left element is greater than the right element.

6. Repeat the passes until the array is sorted in ascending order.

7. Display the sorted array.

8. Pass the sorted array and size to occurrence() using pointers.

9. Inside occurrence(), read the integer target.

10. Initialize first = -1 and last = -1.

11. Traverse the sorted array. When target is found for the first time, store its index in first. Continue searching and update last for every occurrence.

12. If first remains -1, display -1, -1.

13. Otherwise, display the target value and index of the first occurrence and the target value and index of the last occurrence.

14. Stop.

4. C Program

```
#include <stdio.h>

void Bubble_sort(int *nums, int size)

{

  int i, j, temp;

  for (i = 0; i < size - 1; i++)

  {

    for (j = 0; j < size - i - 1; j++)

    {
```

```
        if (*(nums + j) > *(nums + j + 1))
        {
            temp = *(nums + j);
            *(nums + j) = *(nums + j + 1);
            *(nums + j + 1) = temp;
        }
      }
   }
}
void occurrence(int *nums, int size)
{
   int target, first = -1, last = -1, i;
   printf("Enter target: ");
   scanf("%d", &target);
   for (i = 0; i < size; i++)
   {
      if (*(nums + i) == target)
      {
         if (first == -1)
            first = i;
         last = i;
      }
   }
   if (first == -1)
   {
      printf("-1, -1\n");
```

```
    }
    else
    {
        printf("First occurrence: %d at index %d\n", *(nums + first), first);
        printf("Last occurrence: %d at index %d\n", *(nums + last), last);
    }
}
int main()
{
    int size, i;
    printf("Enter size of array: ");
    scanf("%d", &size);
    if (size <= 0)
    {
        printf("Invalid array size.\n");
        return 0;
    }
    int nums[size];
    printf("Enter %d elements: ", size);
    for (i = 0; i < size; i++)
        scanf("%d", &nums[i]);
    Bubble_sort(nums, size);
    printf("Sorted array: ");
    for (i = 0; i < size; i++)
        printf("%d ", nums[i]);
    printf("\n");
```

```
    occurrence(nums, size);

    return 0;

}
```

5. Sample Output

